

Rientr@Returns Project
CNR-STIIMA

Technical Documentation

`rdb2rdf_step2.py`

Matching between relational DB and ontology
via String Matching and NLP models

Version	Step 2
Language	Python 3.10+
Ontology	Rientra.rdf (namespace JobList)
Database	PostgreSQL 17

Contents

1	Introduction and Context	3
1.1	Module Purpose	3
1.2	Position in the Project	3
2	Architecture and Code Structure	4
2.1	Pipeline Overview	4
2.2	File Organization	4
2.3	Dependencies	5
2.3.1	Required Dependencies	5
2.3.2	Optional Dependencies for Full NLP	5
3	Configuration	6
3.1	Global Parameters	6
3.2	Confidence Thresholds	6
4	Data Sources	8
4.1	External Relational Database	8
4.2	Rientra.rdf Ontology	8
4.2.1	Relevant Namespaces	8
4.2.2	O*NET Occupations Present	9
4.2.3	Dynamic Extraction	9
5	Matching Strategies	10
5.1	Strategy 1 – String Matching	10
5.1.1	Description	10
5.1.2	Metrics	10
5.1.3	Implementation	11
5.2	Cosine Similarity – Shared Function	11
5.3	Strategy 2 – BERT base	12
5.3.1	Model	12
5.3.2	Embedding Strategy	12
5.3.3	Limitations: Anisotropy	13
5.4	Strategy 3 – BioBERT	13
5.4.1	Model	13
5.4.2	Reason for Inclusion	13
5.4.3	Observed Results	13
5.5	Strategy 4 – MiniLM	14
5.5.1	Model	14

5.5.2	Differences Compared to Raw BERT	14
5.5.3	Implementation	14
5.5.4	Text Input	15
6	Producing Results in Excel Format	16
6.1	Purpose of Excel Output	16
6.2	Workbook Structure	16
6.3	Detail of the “Matching Results” Sheet	16
6.3.1	Metadata and Columns	16
6.3.2	Verdict Logic	17
6.4	Styles and Conditional Formatting	17
6.5	Implementation Example	18
7	Output and Results Interpretation	19
7.1	Comparative Table	19
7.2	Current Results	19
8	Known Limitations and Future Developments	21
8.1	Current Limitations	21
8.1.1	Insufficient Ontological Vocabulary	21
8.1.2	Uncalibrated BERT Thresholds	21
8.1.3	BioBERT – Wrong Domain	21
8.2	Planned Developments	21
9	Usage Guide	22
9.1	Prerequisites	22
9.2	Installation	22
9.3	Execution	22
9.4	Expected Output	22
A	Known Warning Messages	24
B	Glossary	25

Chapter 1

Introduction and Context

1.1 Module Purpose

The `rdb2rdf_step2.py` module constitutes the second step of the RDB2RDF process of the **Rientr@Returns** project, developed at CNR-STIIMA. The objective is to resolve the *mismatch* between the job labels found in an external relational database (`ext_job`) and the occupations represented in the main Rientr@ ontology, built on the O*NET vocabulary.

The problem can be formally defined as an *entity linking* task: given an entity (job title + description) from a heterogeneous source, find the corresponding ontological class in the reference ontology, considering that names may be completely different while describing the same occupation.

Note

Concrete mismatch example: the job title “*Manager of the digital archive*” in the relational DB corresponds to the O*NET class “*Archivists*” in the ontology. The two names share no words, but the description of activities is semantically equivalent.

1.2 Position in the Project

The module sits between two phases:

- **Upstream:** the `ext_job` table in PostgreSQL, created with `rientra_ext_jobs.sql`, containing 9 occupations with title and description.
- **Downstream:** an excel file (`rientra_matching.xlsx`), which display the jobs from the db with the matched jobs from the ontology, it also display the results of each strategy used to find the match.

Chapter 2

Architecture and Code Structure

2.1 Pipeline Overview

The module executes the following phases in sequence:

1. **Setup:** connection to PostgreSQL, loading of the RDF ontology, dynamic extraction of O*NET occupations.
2. **DB Read:** retrieval of jobs from the `ext_job` table.
3. **Strategy 1:** Lexical String Matching (Jaccard + Overlap + Levenshtein).
4. **Strategy 2:** NLP with base BERT, [CLS] embedding, cosine similarity.
5. **Strategy 3:** NLP with BioBERT, [CLS] embedding, cosine similarity.
6. **Strategy 4:** NLP with MiniLM via `sentence-transformers`, cosine similarity.
7. **Comparison:** comparative table of results with agreement verdict among models.
8. **Excel Output:** creation of the excel file to show the matching jobs.

2.2 File Organization

The file is organized into logical sections separated by delimiter comments. The following table summarizes the main functions with their roles.

Function	Approx. Lines	Role
<code>score_color()</code>	87–93	Maps score → rich style
<code>score_bar()</code>	94–96	Generates ASCII text bar
<code>get_connection()</code>	103–104	Opens PostgreSQL connection
<code>fetch_all()</code>	106–110	Executes query, returns dict list
<code>test_connection()</code>	112–122	Verifies DB connectivity
<code>extract_ontology_jobs()</code>	129–160	Extracts occupations from ontology
<code>_build_onto_text()</code>	162–169	Builds composite O*NET text
<code>_norm()</code> , <code>_jaccard()</code> ,	176–201	String matching metrics

Function	Approx. Lines	Role
<code>_overlap()</code> , <code>_levenshtein()</code> <code>_combined()</code>	200–201	Weighted S1 score
<code>s1_match_one()</code>	203–215	Single job match with S1
<code>run_string_matching()</code>	217–227	Runs S1 on all jobs
<code>cosine_similarity_matrix()</code>	234–253	Vectorized cosine similarity
<code>get_bert_embedding()</code>	260–288	[CLS] embedding with raw BERT
<code>run_bert_matching()</code>	291–349	Matching with BERT base or BioBERT
<code>run_minilm_matching()</code>	356–407	Matching with MiniLM
<code>print_ontology_table()</code>	414–425	Rich table of ontology occupations
<code>print_db_table()</code>	428–438	Rich table of jobs in the DB
<code>print_full_comparison()</code>	456–559	Comparative strategies table
<code>print_summary_multi()</code>	609–652	Summary panel
<code>set_header()</code>	679–686	Generate the setting
<code>style_cell()</code>	689–694	Style
<code>save_excel()</code>	697–935	Save the excel file
<code>main()</code>	722–847	Pipeline orchestration

Table 2.1: Main functions of the module

2.3 Dependencies

2.3.1 Required Dependencies

```
1 pip install psycpg2-binary rdflib scikit-learn rich numpy
```

Listing 2.1: Minimum dependencies installation

2.3.2 Optional Dependencies for Full NLP

```
1 pip install sentence-transformers transformers torch
```

Listing 2.2: NLP dependencies installation

Note

The module implements *graceful degradation*: if `transformers` or `sentence-transformers` are not installed, the related strategies are silently skipped and the output contains only the available strategies.

Chapter 3

Configuration

3.1 Global Parameters

All modifiable parameters are collected in the configuration section at the top of the file, so that no changes to the functional code are required to adapt the module to a different environment.

```
1 DB_CONFIG = {
2     "host":      "localhost",
3     "port":      5432,
4     "dbname":    "rientra_db",
5     "user":      "postgres",
6     "password":  "postgres",
7 }
8
9 ONTOLOGY_FILE = "Rientra.rdf"
10 OUTPUT_DIR    = "output"
11
12 JOBLIST = Namespace("http://www.stiima.cnr.it/JobList#")
13 RIENTRA = Namespace("https://www.stiima.cnr.it/rientra#")
14
15 S1_THRESHOLD    = 0.12
16 BERT_THRESHOLD  = 0.70
17 MINILM_THRESHOLD = 0.50
18
19 MODELS = {
20     "S2_BERT":      "bert-base-uncased",
21     "S3_BioBERT":   "dmis-lab/biobert-base-cased-v1.2",
22     "S4_MiniLM":    "all-MiniLM-L6-v2",
23 }
```

Listing 3.1: Configuration section

3.2 Confidence Thresholds

Thresholds control when a match is considered valid. A match with a score below the threshold is saved as `score_raw` in the visual output but **is not inserted in the RDF graph** as a linking triple.

Strategy	Parameter	Current Value	Notes
S1 String Match	S1_THRESHOLD	0.12	Low: metrics already combined
S2 BERT base	BERT_THRESHOLD	0.70	Raise to 0.92 (anisotropy)
S3 BioBERT	BERT_THRESHOLD	0.70	Raise to 0.92 (anisotropy)
S4 MiniLM	MINILM_THRESHOLD	0.50	Calibrated, reliable

Table 3.1: Configured confidence thresholds

Warning

The BERT base and BioBERT thresholds are currently set to 0.70, but experimental results show that with these models all 9 jobs exceed the threshold regardless of match quality, due to the *anisotropy* phenomenon (see Section 5.3.3). The value recommended by the project supervisor is ≥ 0.90 .

Chapter 4

Data Sources

4.1 External Relational Database

The `ext_job` table in PostgreSQL is the source of jobs to be matched. It has an intentionally minimal structure: three columns representing the identifier, title, and description of the job.

```
1 CREATE TABLE ext_job (  
2     id          VARCHAR(6) PRIMARY KEY, -- ext01, ext02, ...  
3     title       TEXT      NOT NULL,  
4     description TEXT  
5 );
```

Listing 4.1: Schema of the `ext_job` table

The 9 jobs currently present span very different domains: accounting, web content management, data science, digital public relations, secretarial operations and retail. This heterogeneity is intentional and serves to test the limits of the current ontological vocabulary.

4.2 Rientra.rdf Ontology

The ontology is read from the `Rientra.rdf` file in RDF/XML format. It contains 12,250 triples that define the T-Box (classes, properties, relations) of the Rientr@ model.

4.2.1 Relevant Namespaces

Prefix	URI
JobList:	http://www.stiima.cnr.it/JobList#
rientra:	https://www.stiima.cnr.it/rientra#
01:	http://www.stiima.cnr.it/01:

Table 4.1: Main ontology namespaces

4.2.2 O*NET Occupations Present

The ontology currently contains 13 occupations in the `JobList` namespace, extracted from the O*NET classification:

Local URI	O*NET Label
<code>Billing_cost_and_rate_clerks</code>	Billing, Cost, and Rate Clerks
<code>Bookkeeping_accounting_and_auditing_clerks</code>	Bookkeeping, Accounting, Auditing Clerks
<code>Cabinet_makers_and_bench_carpenters</code>	Cabinetmakers and Bench Carpenters
<code>Construction_laborers</code>	Construction Laborers
<code>File_clerks</code>	File Clerks
<code>Gem_and_diamond_workers</code>	Gem and Diamond Workers
<code>Insurance_claims_clerks</code>	Insurance Claims Clerks
<code>Insurance_policy_processing_clerks</code>	Insurance Policy Processing Clerks
<code>Landscaping_and_Groundskeeping_Workers</code>	Landscaping and Groundskeeping Workers
<code>Postal_service_clerks</code>	Postal Service Clerks
<code>Receptionist_and_information_clerks</code>	Receptionists and Information Clerks
<code>Travel_guides</code>	Travel Guides
<code>Word_Processors_and_Typists</code>	Word Processors and Typists

Table 4.2: O*NET occupations present in the ontology

4.2.3 Dynamic Extraction

Occupations are not hardcoded in the script, but are extracted dynamically from the ontology via the `extract_ontology_jobs()` function. For each class in the `JobList` namespace, three predicates are collected:

- `rdfs:label` – canonical name of the occupation
- `01:Net_job_titles` – alternative titles reported by O*NET (e.g. “Accounting Clerk, Accounting Specialist, ...”)
- `01:Net_short_description` – concise description of the work activity

The prefix “Sample of reported job titles:” is automatically removed from `Net_job_titles` before matching, as otherwise descriptive text fragments would end up in the lexical comparison causing spurious matches.

Chapter 5

Matching Strategies

5.1 Strategy 1 – String Matching

5.1.1 Description

Strategy 1 is a lexical baseline that does not use language models. It compares the **job title in the DB** with the canonical label and all alternative O*NET titles of each occupation in the ontology, using three linearly combined metrics.

Note

Strategy 1 uses **only the title** of the job, not the description. This makes it fast but insensitive to the semantic content of work activities.

5.1.2 Metrics

Jaccard Similarity

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (5.1)$$

where A and B are the sets of words (after lowercase normalization) of the two titles. Measures the proportion of shared words over total distinct words. Insensitive to word order.

Overlap Coefficient

$$O(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)} \quad (5.2)$$

Divides by the cardinality of the smaller set rather than the union. It is more generous than Jaccard when a short title is semantically contained in a longer one (e.g. “Data entry” contained in “Data Entry Operator”).

Normalized Levenshtein Similarity

$$L(s_1, s_2) = 1 - \frac{d_{edit}(s_1, s_2)}{\max(|s_1|, |s_2|)} \quad (5.3)$$

where d_{edit} is the edit distance (minimum number of insertions, deletions and substitutions of characters). It is the only one of the three metrics sensitive to character order – useful for abbreviations and spelling variants.

Combined Score

$$\text{score}_{S1}(a, b) = 0.5 \cdot J(a, b) + 0.3 \cdot O(a, b) + 0.2 \cdot L(a, b) \quad (5.4)$$

The weights give priority to Jaccard as the main metric, with Overlap as support for length asymmetries and Levenshtein for spelling variations.

5.1.3 Implementation

```

1 def s1_match_one(title: str, onto_jobs: list[dict]) -> dict | None:
2     best, bj, bv = 0.0, None, None
3     for j in onto_jobs:
4         cand = [j["label"]]
5         if j["titles"]:
6             cand += [t.strip() for t in j["titles"].split(",") if
7                     t.strip()]
8         for c in cand:
9             sc = _combined(title, c)
10            if sc > best:
11                best, bj, bv = sc, j, c
12 if best >= S1_THRESHOLD:
13     return {"job": bj, "score": best, "via": bv}
14 return None

```

Listing 5.1: S1 matching function

The "via" key in the returned dictionary indicates exactly which specific label (among the alternative O*NET ones) produced the maximum score – useful information for debugging and for evaluating match quality.

5.2 Cosine Similarity – Shared Function

All three NLP strategies use the same function to calculate cosine similarity, ensuring methodological uniformity in comparisons.

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} \quad (5.5)$$

where $\mathbf{A}$ is the embedding vector of the DB job and $\mathbf{B}$ is the embedding vector of an O*NET occupation. The result is between -1 and 1 ; for language model embeddings the effective range is $[0, 1]$.

```

1 def cosine_similarity_matrix(query_emb: np.ndarray,
2                             corpus_embs: np.ndarray) -> np.ndarray:
3     query_norm = query_emb / (np.linalg.norm(query_emb) + 1e-9)
4     corpus_norm = corpus_embs / (
5         np.linalg.norm(corpus_embs, axis=1, keepdims=True) + 1e-9
6     )
7     return corpus_norm.dot(query_norm)

```

Listing 5.2: Vectorized implementation of cosine similarity

The $+10^{-9}$ term (numerical stability epsilon) prevents division by zero in the theoretical case where a vector has zero norm. The vectorized form computes in a single operation the similarity between the query vector and all 13 O*NET vectors, without Python loops.

5.3 Strategy 2 – BERT base

5.3.1 Model

Parameter	Value
HuggingFace identifier	<code>bert-base-uncased</code>
Number of parameters	110 million
Architecture	Transformer encoder, 12 layers
Embedding dimension	768
Pre-training	Wikipedia EN + BookCorpus
Pre-training tasks	Masked LM + Next Sentence Prediction

Table 5.1: Characteristics of bert-base-uncased

5.3.2 Embedding Strategy

BERT does not natively produce a sentence embedding. To derive it, the vector of the special [CLS] token is used, inserted automatically at the beginning of each input sequence. This token was designed during pre-training to capture an aggregated representation of the entire sequence.

```

1 def get_bert_embedding(text, tokenizer, model, max_length=512):
2     inputs = tokenizer(text, return_tensors="pt",
3                       truncation=True, max_length=max_length,
4                       padding=True)
5     with torch.no_grad():
6         outputs = model(**inputs)
7     # shape: [batch=1, seq_len, 768] -> [768]
8     cls_embedding = outputs.last_hidden_state[:, 0,
9     :].squeeze().numpy()
10    return cls_embedding

```

Listing 5.3: embedding extraction from BERT

`torch.no_grad()` disables gradient computation since the model is used only for inference, reducing memory usage.

5.3.3 Limitations: Anisotropy

BERT has not been fine-tuned for sentence similarity. Its embeddings tend to occupy a restricted area of the vector space (a phenomenon known as *anisotropy*): vectors all point in similar directions, making cosine similarity systematically high regardless of actual semantic similarity. In practice, with a threshold of 0.70, all 9 jobs in the dataset get a match – which is not necessarily correct.

5.4 Strategy 3 – BioBERT

5.4.1 Model

Parameter	Value
HuggingFace identifier	<code>dmis-lab/biobert-base-cased-v1.2</code>
Number of parameters	110 million
Architecture	Transformer encoder, 12 layers (like BERT)
Embedding dimension	768
Pre-training	BERT base + PubMed abstracts + PMC full text
Additional corpus	4.5 billion words of biomedical literature

Table 5.2: Characteristics of BioBERT

5.4.2 Reason for Inclusion

BioBERT is included for **methodological comparison**, not because it is the best model for this task. Its presence allows empirical observation of how domain specialization influences results: a model trained on medical literature produces different embeddings from base BERT when processing texts in the administrative/occupational domain.

5.4.3 Observed Results

Tests show that BioBERT maps 8 out of 9 jobs to the same class (*File Clerks*), regardless of content. This behavior is attributable to the mismatch between the training domain (biomedical literature) and the task domain (job matching). The *File Clerks* class has a short and generic O*NET description that produces a “neutral” vector close to many texts in a semantic space not calibrated for this domain.

Warning

Analysis of BioBERT’s architectural characteristics, combined with the poor performance detected during the experimental phase, suggests that its use is ineffective in the present case. Unlike what was observed with the standard BERT model, the specificity of BioBERT’s pre-training on biomedical domains did not yield the expected benefits, rendering any further fine-tuning activity superfluous and unproductive.

5.5 Strategy 4 – MiniLM

5.5.1 Model

Parameter	Value
HuggingFace identifier	sentence-transformers/all-MiniLM-L6-v2
Number of parameters	22 million
Architecture	Transformer encoder, 6 layers (distilled from BERT)
Embedding dimension	384
Fine-tuning	Sentence similarity on 1+ billion pairs
Library	sentence-transformers

Table 5.3: Characteristics of MiniLM

5.5.2 Differences Compared to Raw BERT

MiniLM was trained with an explicit sentence similarity objective via *Siamese* architecture: two networks with shared weights process pairs of sentences and are optimized to produce vectors close together for semantically similar sentences and vectors far apart for semantically dissimilar sentences. The result is that its cosine similarity scores are **calibrated**: a score of 0.86 indicates genuine semantic similarity, a score of 0.30 indicates genuine dissimilarity.

5.5.3 Implementation

```

1 model = SentenceTransformer("all-MiniLM-L6-v2")
2 onto_embs = model.encode(onto_texts, convert_to_numpy=True)
3
4 for i, row in enumerate(db_jobs):
5     q_emb = model.encode(db_texts[i], convert_to_numpy=True)
6     # Cosine similarity computed with the same function as S2/S3
7     sims = cosine_similarity_matrix(q_emb, onto_embs)
8     idx = int(np.argmax(sims))
9     sc = float(sims[idx])
10    m = {"job": onto_jobs[idx], "score": sc} if sc >= threshold else
        None

```

Listing 5.4: Matching with MiniLM

O*NET embeddings are pre-computed only once before the loop over DB jobs, optimizing performance. Although MiniLM provides cosine similarity functions internally, the `cosine_similarity_matrix()` function is used explicitly for uniformity with S2 and S3.

5.5.4 Text Input

Both for DB jobs and for O*NET occupations, the text passed to the model is a concatenation of multiple fields:

- **DB Job:** title + ". " + description
- **O*NET Occupation:** label + Net_job_titles + Net_short_description

The more semantic context is provided, the more precise the embeddings produced by the model.

Chapter 6

Producing Results in Excel Format

6.1 Purpose of Excel Output

Unlike the previous phase, the current module does not generate an RDF A-Box graph, but produces an analytical report in Microsoft Excel format (.xlsx). This choice is motivated by the need to validate confidence thresholds and compare the various matching strategies (lexical and neural) in a readable and structured way before definitive persistence in the Knowledge Graph.

The file is automatically generated in the `output/` folder with the name `rientra_matching.xlsx`.

6.2 Workbook Structure

The Excel document is organized into three distinct worksheets, each with a specific role in the results evaluation pipeline:

1. **Matching Results:** Main sheet containing the results of all matching strategies for each database job.
2. **Ontology Jobs:** List of O*NET occupations extracted from the original ontology, used as reference.
3. **Score Detail:** Detailed matrix reporting raw scores for each job-model combination.

6.3 Detail of the “Matching Results” Sheet

The main sheet collects information from the PostgreSQL database and enriches it with NLP model results.

6.3.1 Metadata and Columns

For each row (corresponding to a DB job), the following are reported:

- **DB Data:** ID (`ext_job`), Title and Description.
- **Individual IRI:** The URI that would identify the entity in the graph (`rientra:ExtJob_*`).

- **S1 String Match:** Label found, score and specific label used for the match.
- **NLP Strategies (S2, S3, S4):** Identified occupation, *cosine similarity* score and URI of the corresponding class.

6.3.2 Verdict Logic

The last column of the sheet provides a synthetic **Verdict** based on consensus among NLP models.

Verdict	Description
All agree	All active NLP models identified the same O*NET URI.
Partial agreement	There is consensus among some models or the score is considered reliable.
Diverge	NLP models point to different O*NET occupations.
S1 only	Only string matching produced a result above the threshold.
No match	No strategy exceeded the configured confidence thresholds.

Table 6.1: Verdict logic in the Excel sheet

6.4 Styles and Conditional Formatting

To facilitate visual analysis, the module applies specific styles based on score quality:

- **Coloring by Strategy:** Each matching column has a distinctive color (e.g. yellow for S1, green for MiniLM).
- **Score Highlighting:** Scores above the thresholds (≥ 0.70 for BERT and ≥ 0.50 for MiniLM) are highlighted in bold green.
- **Below Threshold:** Insufficient results report the *raw* value in red with the label “below threshold”.

6.5 Implementation Example

Below is the structure of the Python function that uses the `openpyxl` library:

```
1 def save_excel(db_jobs, s1_res, nlp_results, model_names, onto_jobs):
2     wb = Workbook()
3     ws1 = wb.active
4     ws1.title = "Matching Results"
5     # Header and style configuration
6     for r_idx, row in enumerate(db_jobs, start=2):
7         # Writing DB data and strategy results
8         # Computing consensus verdict
9     wb.save(os.path.join(OUTPUT_DIR, "rientra_matching.xlsx"))
```

Listing 6.1: Snippet of the `save_excel` function

Chapter 7

Output and Results Interpretation

7.1 Comparative Table

Section 6 of the output shows a table with one row per DB job and a pair of columns (match + score) for each available strategy. The **Verdict** column summarizes the agreement among NLP models:

Verdict	Meaning	Reliability
✓ all	All NLP models agree	High
~ part.	Partial agreement (some models)	Medium
≠ div.	Models point to different occupations	Low
S1 only	Only string matching has a match	Check manually
× none	No match above threshold	Job not in ontology

Table 7.1: Interpretation of the Verdict column

Note

The verdict is computed by comparing the URIs of the occupations found, not the scores. Two models with very different scores (e.g. 0.929 and 0.863) that point to the same occupation give verdict “✓ all”. Two models with similar scores that point to different occupations give “≠ div.”.

7.2 Current Results

With the 13 occupations currently in the ontology and the configured thresholds:

Job	S4 MiniLM	Match	Reliability
ext01 Accounting clerk	0.863	Bookkeeping Clerks	High
ext05 Data entry employee	0.556	Bookkeeping Clerks	Debatable
ext09 Supermarket cashier	0.511	Billing Clerks	Debatable
ext02–04, 06–08	< 0.50	No match	Occupation absent

Table 7.2: MiniLM results with threshold 0.50

The only semantically reliable match is **ext01**. The other 6 jobs find no correspondence because their occupations (Data analyst, Data scientist, Content manager web, etc.) are not among the 13 classes in the ontology.

Chapter 8

Known Limitations and Future Developments

8.1 Current Limitations

8.1.1 Insufficient Ontological Vocabulary

The main bottleneck is not the quality of the NLP models but the limited coverage of the O*NET vocabulary in the ontology: 13 occupations out of thousands available. Adding the missing occupations (Data analyst, Data scientist, Content manager, etc.) will drastically improve MiniLM results.

8.1.2 Uncalibrated BERT Thresholds

With a threshold of 0.70, BERT and BioBERT find a match for all 9 jobs regardless of quality. The threshold needs to be raised to ≥ 0.90 as recommended by the project supervisor.

8.1.3 BioBERT – Wrong Domain

BioBERT was included for methodological comparison but is not suitable for the job matching task. It produces unreliable results (8/9 jobs mapped to *File Clerks*) and could be replaced with a more appropriate model for the occupational domain.

8.2 Planned Developments

1. **Threshold Update:** raise BERT_THRESHOLD to 0.92 and add the “gap” column (difference between first and second score) as a measure of match robustness.
2. **Ontology Enrichment:** add the 10 new O*NET occupations covering the jobs currently without a match.
3. **Modified Model:** The decision has been made to move beyond static mapping between the O*NET and ESCO frameworks, opting instead for the implementation of a BERT model trained on real datasets. This approach is aimed at accurately identifying equivalences and semantic similarities between profiles.

Chapter 9

Usage Guide

9.1 Prerequisites

1. PostgreSQL 17 running and accessible on localhost:5432
2. rientra_db database created: `createdb -U postgres rientra_db`
3. ext_job table populated: `psql -U postgres -d rientra_db -f rientra_ext_jobs.sql`
4. Rientra.rdf file in the same directory as the script
5. Python virtual environment activated

9.2 Installation

```
1 python3 -m venv venv
2 source venv/bin/activate           # macOS/Linux
3 # or: venv\Scripts\activate       # Windows
4
5 pip install psycopg2-binary rdflib scikit-learn rich numpy
6 pip install transformers torch sentence-transformers
```

Listing 9.1: Environment setup

9.3 Execution

```
1 python3 rdb2rdf_step2.py #macOS
2
3 python rdb2rdf_step2.py # Windows
```

Listing 9.2: Running the script

9.4 Expected Output

At the end of execution, the following is produced in the output/ folder:

- rientra_matching.xlsx – excel file

Complete execution with all NLP models takes approximately 2–5 minutes on the first run (downloading models from HuggingFace) and approximately 30–60 seconds on subsequent runs (local cache).

Appendix A

Known Warning Messages

During execution, some warning messages appear that are normal and do not indicate errors:

Message	Cause and Behavior
<code>cls.predictions.* UNEXPECTED</code>	Pre-training layers (Masked LM) not used. Discarded without effect on functionality.
<code>cls.seq_relationship.* UNEXPECTED</code>	Next Sentence Prediction layer not used. Discarded.
<code>embeddings.position_ids UNEXPECTED</code>	Buffer deprecated in recent versions of Transformers. Ignored.
Unauthenticated requests to HF Hub	Models downloaded without HuggingFace token. Works, but with more restrictive rate limiting. Solvable by setting <code>HF_TOKEN</code> .

Table A.1: Known warnings and their interpretation

Appendix B

Glossary

A-Box (Assertional Box)

The part of the RDF graph containing individuals (instances), in contrast to the T-Box which contains class and property definitions.

Anisotropy

A phenomenon whereby embeddings produced by raw BERT tend to occupy a restricted region of the vector space, with all directions similar to each other. Causes systematically high cosine similarity scores for any pair of texts.

CLS token

Special [CLS] token inserted by BERT at the beginning of each input sequence. Its vector in the last hidden state is used as the embedding of the entire sequence.

Cosine Similarity

A measure of similarity between two vectors based on the angle they form, independent of their magnitude.

Embedding

Dense vector representation of a text in a high-dimensional numerical space, produced by a neural model. Semantically similar texts have nearby embeddings.

Entity Linking

NLP task consisting of linking textual mentions of entities to corresponding entries in a structured knowledge base.

Fine-tuning

Process of re-training a pre-trained model on a specific dataset for a specific task, partially updating the model's weights.

Knowledge Graph

Structured graph representing entities and relationships between them in the form of subject-predicate-object triples.

O*NET

Occupational Information Network: a database of the US Department of Labor providing standardized descriptions of thousands of occupations, including skills, activities, and alternative titles.

RDB2RDF

W3C standard for mapping relational data (SQL databases) to RDF data. Includes Direct Mapping and R2RML.

T-Box (Terminological Box)

The part of the RDF graph (or OWL ontology) containing class definitions, properties and axioms – in contrast to the A-Box which contains individuals.

RDF Triple

Atomic unit of the RDF model, in the form *subject – predicate – object*.